A HYBRID AI-BASED INTRUSION DETECTION AND AUTOMATED RESPONSE SYSTEM FOR NETWORK SECURITY

A PROJECT REPORT

Submitted by

AJAY KUMAR S	**811722243002**
DINESH M	**811722243014**
VIGNESH S	**811722243053**

in partial fulfillment of the requirements for the award degree of

Bachelor of Technology

20AI8201 PROJECT WORK

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM – 621112

MARCH 2026

A HYBRID AI-BASED INTRUSION DETECTION AND AUTOMATED RESPONSE SYSTEM FOR NETWORK SECURITY

A PROJECT REPORT

Submitted by

AJAY KUMAR S	**811722243002**
DINESH M	**811722243014**
VIGNESH S	**811722243053**

in partial fulfillment of the requirements for the award degree of

Bachelor of Technology

20AI8201 PROJECT WORK

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM – 621112

MARCH 2026

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM - 621112

BONAFIDE CERTIFICATE

The work embodied in the present project report entitled "**A HYBRID AI BASED INTRUSION DETECTION AND AUTOMATED RESPONSE SYSTEM FOR NETWORK SECURITY**" has been carried out by the students **AJAY KUMAR S, DINESH M, VIGNESH S .** The work reported herein is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for assessment.

Date of Viva Voce: ……

Mrs. S.GEETHA M.E.,
SUPERVISOR
Assistant Professor
Department of Artificial Intelligence and Data Science
K. Ramakrishnan College of Technology(Autonomous)
Samayapuram – 621 112

Dr. T.AVUDAIAPPAN M.E.,Ph.D.,
HEAD OF THE DEPARTMENT
Professor
Department of Artificial Intelligence
K. Ramakrishnan College of Technology(Autonomous)
Samayapuram – 621 112

INTERNAL EXAMINER **EXTERNAL EXAMNIER**

ACKNOWLEDGEMENT

We thank our **Dr. N.Vasudevan**, Principal, for his valuable suggestions and support during the course of our research work.

We thank our **Dr. T.Avudaiappan, M.E., Ph.D.,** Head of the Department, Professor, Department of Artificial Intelligence for his valuable suggestions and support during the course of our research work.

We wish to record our deep sense of gratitude and profound thanks to our Guide **Mrs. S. Geetha, M.E.,** Assistant Professor, Department of Artificial Intelligence and Data Science for her keen interest, inspiring guidance, constant encouragement with our work during all stages, to bring this thesis into fruition.

We are extremely indebted to our project coordinator **Ms. R. Swetha Barathi M.E,** Assistant Professor, Department of Artificial Intelligence and Data Science, for her valuable suggestions and support during the course of our research work.

We also thank the faculty and non-teaching staff members of the Artificial Intelligence, K. Ramakrishnan College of Technology (Autonomous), Samayapuram for their valuable support throughout the course of our research work.

Finally, we thank our parents, friends and our well wishes for their kind support.

SIGNATURE

AJAY KUMAR S

DINESH M

VIGNESH S

ABSTRACT

A Hybrid AI-Based Intrusion Detection and Automated Response System for Network Security enhances cybersecurity by detecting malicious network activities in real time. The system integrates deep learning, machine learning, and reinforcement learning to improve intrusion detection and automate response actions. It employs an Autoencoder for anomaly detection, an XGBoost classifier for attack classification, and a PPO reinforcement learning agent to trigger actions such as allow, block, alert, or quarantine, ensuring efficient and scalable network protection.The system is evaluated using the CIC-IDS2017 dataset and achieves high accuracy with a low false positive rate. It also includes real-time monitoring, alert generation, and incident reporting features. The proposed approach provides an efficient, scalable, and intelligent solution for proactive cyber threat detection and mitigation. The system is designed to handle large-scale network environments with high efficiency.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
CNN	-	Convolutional Neural Network
DDoS	-	Distributed Denial of Service
XGBoost	-	Extreme Gradient Boosting
PPO	-	Proximal Policy Optimization
IDS	-	Intrusion Detection System
DRL	-	Deep Reinforcement Learning
GPU	-	Graphics Processing Unit
IDE	-	Integrated Development Environment

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The AI-Driven Intrusion Detection System (AI-IDS) is developed to enhance network security by detecting and responding to malicious network activities in real time. With the rapid growth of digital communication and internet-based services, cyber threats such as Distributed Denial of Service (DDoS), port scanning, brute force attacks, and botnet activities have become increasingly common. Traditional intrusion detection systems mainly rely on signature-based detection techniques, which are often ineffective in identifying new or unknown threats. Therefore, intelligent security systems that can automatically detect and respond to attacks are essential.

The proposed system integrates deep learning, machine learning, and reinforcement learning techniques to improve threat detection accuracy and automate response actions. A deep autoencoder model is used for anomaly detection by learning normal network traffic patterns and identifying unusual behavior. An XGBoost classifier is applied to classify network traffic as benign or malicious. In addition, a Proximal Policy Optimization (PPO) reinforcement learning agent is used to determine appropriate response actions such as allowing traffic, blocking suspicious activities, generating alerts, or quarantining threats. The system also provides real-time monitoring, automated alerts, and security reporting, making it a scalable and intelligent solution for modern cybersecurity challenges.

1.2 OBJECTIVE

A Hybrid AI-Based Intrusion Detection and Automated Response System for Network Security aims to design and develop an intelligent framework capable of detecting malicious network activities in real time. With the rapid expansion of internet usage and digital communication, networks are increasingly exposed to sophisticated cyber threats such as Distributed Denial of Service (DDoS), port scanning, brute force attacks, and botnet intrusions. Traditional intrusion detection systems mainly rely on signature-based or rule-based approaches, which are often ineffective in detecting unknown or evolving threats. These systems also generate high false positives and lack adaptability. Therefore, there is a need for an advanced, intelligent system that can continuously monitor network traffic, analyze patterns, and accurately identify suspicious behavior to improve overall network security.

The project further focuses on integrating advanced artificial intelligence techniques, including deep learning, machine learning, and reinforcement learning, to enhance detection accuracy and efficiency. An Autoencoder model is utilized to learn normal network traffic behavior and detect anomalies, while an XGBoost classifier is applied to classify traffic as benign or malicious based on extracted features. This hybrid approach combines the strengths of both unsupervised and supervised learning methods, enabling the system to detect both known and unknown cyber attacks effectively while reducing false alarms. Additionally, the system aims to implement automated response mechanisms using reinforcement learning. A PPO-based reinforcement learning agent selects optimal actions such as allowing traffic, blocking suspicious activities, generating alerts, or quarantining threats. This intelligent decision-making process minimizes manual intervention, improves response time, and ensures real-time protection. Overall, the system provides a scalable, adaptive, and efficient solution for modern network security challenges.

CHAPTER 2

LITERATURE SURVEY

2.1 ANOMALY-BASED INTRUSION DETECTION MODEL USING DEEP LEARNING FOR IOT NETWORKS

Muaadh Abdulfattah Alsoufi, M. M.

Siraj Published year: 2025

This study proposes an anomaly-based intrusion detection system designed specifically for Internet of Things (IoT) networks using deep learning techniques. The authors introduce a framework that employs a Sparse Autoencoder (SAE) to learn representations of network traffic and reduce high-dimensional feature spaces. The proposed approach addresses limitations of traditional machine-learning intrusion detection methods by leveraging deep learning to improve anomaly detection performance in heterogeneous IoT environments. The framework demonstrates improved capability in identifying malicious activities within IoT network traffic.

Merits

- Sparse Autoencoder effectively learns data representations and reduces dimensionality.
- Capable of detecting anomalous traffic patterns in IoT networks.

Demerits

- IoT environments often involve resource-constrained devices that may struggle with computationally intensive deep learning models.
- Performance may degrade in highly heterogeneous IoT networks with diverse traffic patterns.

2.2 DEEP Q-LEARNING BASED REINFORCEMENT LEARNING APPROACH FOR NETWORK INTRUSION DETECTION

Hooman Alavizadeh, Julian Jang-

Jaccard Published year: 2024

This paper introduces a reinforcement learning-based network intrusion detection system using Deep Q-Learning (DQL). The model integrates Q-learning with a deep feed-forward neural network to enable automated detection of cyberattacks. Experimental evaluations using benchmark datasets demonstrate that the proposed model can effectively detect multiple intrusion classes and outperform several conventional machine learning approaches. Additionally, the model adapts dynamically to changing network environments by continuously updating its learning policy. However, the approach requires significant computational resources and careful tuning of hyperparameters. Overall, the study highlights the potential of reinforcement learning in enhancing intelligent and adaptive intrusion detection systems.

Merits

- Reinforcement learning enables adaptive and autonomous learning.
- Capable of continuously improving detection policies through experience.
- Achieves competitive detection accuracy compared to traditional models.

Demerits

- Training deep reinforcement learning models requires high computational resources.
- Performance depends heavily on hyperparameter tuning and training environment design.

2.3 A MICRO REINFORCEMENT LEARNING ARCHITECTURE FOR INTRUSION DETECTION SYSTEMS (MRLC)

Darabi et al.

Published year: 2022

This research proposes a micro reinforcement learning architecture designed for intrusion detection systems. The architecture leverages reinforcement learning techniques to improve adaptive security decision-making by learning from previous network traffic experiences. The model focuses on enabling IDS systems to dynamically update their detection strategies by analyzing historical patterns of attacks and normal traffic. The model also supports continuous learning, enabling better handling of evolving cyber threats. However, the architecture may face scalability issues when applied to large-scale networks with high traffic volumes. Overall, the study demonstrates the effectiveness of reinforcement learning in building adaptive and intelligent intrusion detection systems.

Merits:

- Reinforcement learning enables adaptive intrusion detection.
- Learns from each experience independently, improving detection performance.
- Demonstrates improved results on benchmark IDS datasets.

Demerits:

- Scalability can become challenging in extremely large network environments.
- Reinforcement learning training may require large amounts of interaction data

2.4 BUILDING AUTO-ENCODER INTRUSION DETECTION SYSTEM BASED ON RANDOM FOREST (AE-IDS)

X. K. Li

Published year: 2020

This study proposes a hybrid intrusion detection system that integrates Autoencoder-based anomaly detection, XGBoost classification, and reinforcement learning for automated response. The Autoencoder learns normal network behavior and detects anomalies by identifying deviations in traffic patterns. These extracted features are then passed to an XGBoost classifier, which accurately classifies network traffic as benign or malicious. This combination of unsupervised learning, supervised classification, and adaptive response improves detection accuracy and reduces false positives. The proposed system demonstrates enhanced performance compared to traditional intrusion detection methods by providing real-time detection, scalability, and automated threat mitigation in dynamic network environments..

Merits

- Hybrid architecture combines unsupervised feature learning with supervised classification.
- Improves detection accuracy by extracting meaningful data representations.
- Random Forest provides strong classification capability.

Demerits

- Two-stage pipeline increases computational complexity.
- Final classification still depends on labeled datasets.

2.5 DEEP REINFORCEMENT LEARNING-BASED AUTOENCODER FOR INTRUSION DETECTION

Y. Dai

Published year: 2020

This study proposes a hybrid intrusion detection framework that integrates Convolutional Autoencoder (CAE) with Deep Reinforcement Learning (DRL). The autoencoder performs feature extraction and noise reduction from network traffic data, while the reinforcement learning component dynamically adjusts detection and response strategies. The system aims to provide adaptive security by combining deep representation learning with policy-based decision-making. The model is designed to improve detection of complex attack patterns while maintaining adaptability to evolving network threats.

Merits

- Integrates autoencoder feature learning with reinforcement learning decision- making.
- Capable of adapting to evolving attack patterns.
- Improves representation learning for complex network traffic.

Demerits

- Computationally expensive due to deep learning and reinforcement learning integration.
- Training stability can be challenging when coordinating both components.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Traditional intrusion detection systems rely mainly on signature-based or rule-based detection methods to identify cyber attacks. These systems analyze network traffic and compare it with known attack patterns stored in a database. When a match is found, the system identifies the traffic as malicious.

Although these systems are useful for detecting known attacks, they are less effective against new or evolving threats. They also require manual monitoring and intervention to respond to detected attacks, which may delay mitigation in large-scale network environments.

Furthermore, these systems lack adaptability and cannot learn from new attack patterns automatically. They often generate high false positive rates, reducing reliability in dynamic network conditions. As cyber threats become more sophisticated, traditional IDS fail to provide real-time intelligent detection and automated response mechanisms, highlighting the need for advanced AI-based solutions.

3.1.1 Demerits

- Detects only known attack signatures
- High false positive rates in dynamic network environments
- Unable to detect zero-day or unknown attacks
- Requires manual monitoring and response

3.2 PROPOSED SYSTEM

The proposed system introduces an AI-driven hybrid intrusion detection and automated response framework to improve cybersecurity protection. It combines deep learning, machine learning, and reinforcement learning models to detect malicious network activities and respond automatically. The Autoencoder model learns normal network traffic patterns and identifies anomalies. The XGBoost classifier analyzes network features and classifies traffic as benign or malicious. A PPO reinforcement learning agent processes the outputs of these models and selects the most appropriate action such as allowing traffic, blocking suspicious activity, generating alerts, or quarantining threats.

The system also includes a decision engine for risk assessment and a real-time monitoring dashboard that allows users to observe network activities, alerts, and response actions. Additionally, the system supports real-time data processing, enabling faster detection and response to cyber threats. This approach enhances scalability and adaptability, making the system suitable for modern dynamic network environments.

3.2.1 Merits

- High detection accuracy with reduced false positives
- Ability to detect unknown and evolving cyber attacks
- Automated response actions for faster threat mitigation
- Real-time monitoring dashboard for network activity
- Scalable and intelligent cybersecurity solution

CHAPTER 4

SYSTEM SPECIFICATIONS

4.1 HARDWARE SPECIFICATIONS

- Processor : Intel Core (i5/i7) / AMD Ryzen 7 (or higher)
- RAM : 4 GB or higher
- Hard disk : Minimum 256 GB
- Network : Stable Internet Connection
- Graphics Processing Unit : NVIDIA GTX 1660 (6 GB)

4.2 SOFTWARE SPECIFICATIONS

- Operating system : Windows 10 / 11 (64-bit) or Ubuntu 20.04+
- Front End : React with Tailwind CSS
- Back End : Flask / FastAPI
- AI / ML Libraries : TensorFlow, PyTorch, OpenCV, Librosa
- Data Handling Libraries : NumPy, Pandas
- IDE / Code Editor : Visual Studio Code
- Web Browser : Google Chrome

CHAPTER 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

System architecture defines the overall structure and organization of the proposed system. It illustrates how different components of the system interact with each other to perform specific functions and achieve the project objectives. The architecture provides a clear understanding of the workflow of the system, starting from data collection to threat detection and response. In the proposed AI-Driven Intrusion Detection System (IDS), the architecture is designed to analyze network traffic data, apply machine learning and deep learning techniques, and generate intelligent responses to potential cyber threats in real time.

The proposed AI-Driven Intrusion Detection System consists of several functional layers including data collection, preprocessing, detection, decision-making, response handling, and visualization. Each component plays a crucial role in ensuring accurate detection and efficient handling of network attacks. The architecture begins with the data layer, where the CIC-IDS2017 dataset is used as the primary source of network traffic data. This dataset contains both normal and malicious traffic records with various features representing network behavior. After collecting the data, the system performs preprocessing. In this stage, the dataset is cleaned by removing inconsistencies, handling missing values, and transforming the data into a structured format suitable for machine learning models. Feature scaling and normalization techniques are also applied to improve model performance. Once the data is preprocessed, it is passed to the detection layer. In this stage,

two types of models are used to identify threats. The XGBoost classifier, a supervised learning model, is used to detect known attack patterns based on labeled data, a deep autoencoder, an unsupervised learning model, is used to identify anomalies by analyzing reconstruction errors, which helps in detecting unknown or zero-day attacks. The combination of these models improves the overall accuracy and robustness of the system.

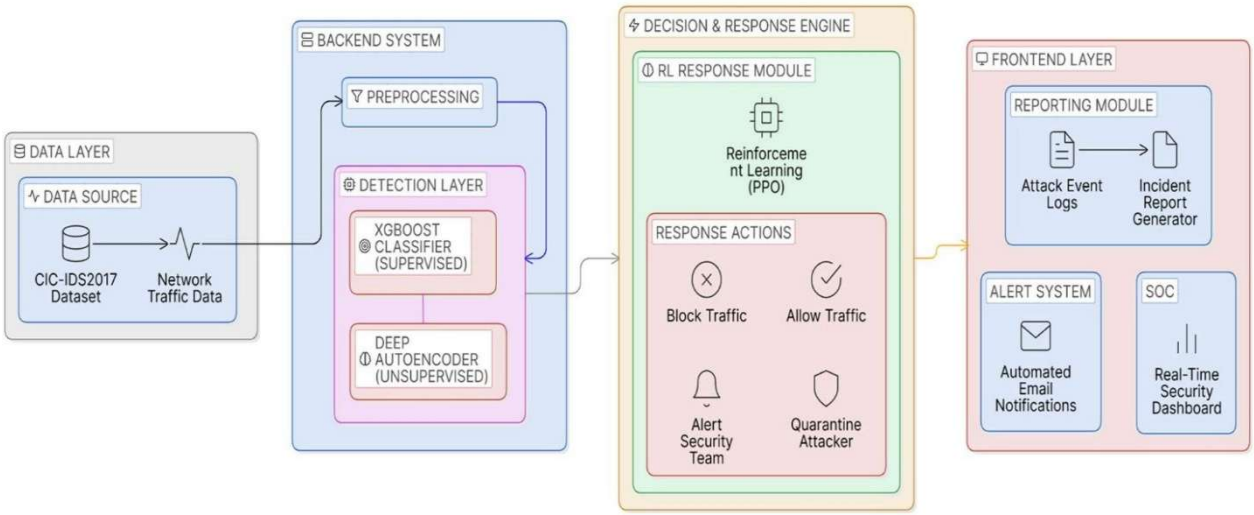

Figure .5.1 System Architecture

After detection, the system moves to the decision and response stage. In this stage, a reinforcement learning agent based on Proximal Policy Optimization (PPO) analyzes the detection results and selects the most appropriate action. The system can response actions such as blocking malicious traffic, allowing normal traffic, alerting the security team, or quarantining the attacker. This intelligent decision- making process enables automated and adaptive threat handling. Finally, the results are presented through the frontend layer. The system allows users to monitor network activity, anomaly scores, and detected threats. Additionally, an alert system sends automated email notifications when attacks are detected..

CHAPTER 6

MODULES DESCRIPTION

6.1 DATA COLLECTION AND PREPROCESSING

The data collection and preprocessing module is the first stage of the proposed intrusion detection system. In this stage, large amounts of network traffic data are gathered for analysis and model training. The system primarily uses the CIC-IDS2017 dataset, which contains both normal network activities and various cyber attacks such as DDoS, PortScan, brute force, and infiltration attacks. Collecting a diverse dataset is important because it allows the system to understand different network behaviors and recognize malicious patterns effectively. This dataset provides realistic traffic scenarios that help train machine learning models to detect threats accurately.

Once the dataset is collected, several preprocessing steps are performed to prepare the data for machine learning algorithms. Data cleaning is carried out to remove missing values, duplicate records, and irrelevant attributes that may affect model performance. Feature selection techniques are applied to identify the most relevant features such as packet size, protocol type, connection duration, and flow statistics. These features play a key role in distinguishing between normal and malicious traffic patterns.The preprocessing stage also includes encoding categorical variables into numerical values and applying normalization techniques to scale the data within a consistent range. Furthermore, categorical features are converted into numerical values using encoding techniques, making the data compatible with machine learning models. Normalization is applied to scale the features into a consistent range, which improves model convergence and accuracy.

6.2 MODEL DEVELOPMENT AND ANALYSIS

The model development and analysis module focuses on designing and implementing intelligent machine learning models to detect cyber attacks in network traffic. In this stage, a hybrid AI-based approach is adopted by combining deep learning and machine learning techniques to improve detection performance. The first component used in the system is an Autoencoder model, which is a type of unsupervised deep learning algorithm designed to learn normal patterns of network behavior. By training on benign traffic, the Autoencoder captures the underlying structure of normal data and identifies deviations as anomalies. These anomalies may indicate suspicious or malicious activities, enabling the system to detect unknown or zero-day attacks effectively.

In addition to anomaly detection, the system employs the XGBoost algorithm for supervised attack classification. XGBoost is a powerful gradient boosting technique that performs efficiently on structured datasets and handles complex relationships between features. It analyzes the selected network attributes and classifies traffic into categories such as benign or specific attack types. This classification provides detailed insights into the nature of detected threats and enhances the overall accuracy of the intrusion detection system. The combination of Autoencoder and XGBoost allows the system to benefit from both anomaly detection and precise classification. During this module, the models are trained using the preprocessed dataset and evaluated using performance metrics such as accuracy, precision, recall, and F1-score. Model analysis is carried out to compare different configurations and optimize parameters for better performance. This ensures that the most efficient and reliable model is selected, improving detection capability, reducing false positives, and enhancing system robustness

6.3 REINFORCEMENT LEARNING RESPONSE

The reinforcement learning response module introduces an intelligent mechanism that enables the system to automatically respond to detected cyber threats in real time. In this module, a reinforcement learning algorithm known as Proximal Policy Optimization (PPO) is utilized to determine the most appropriate response actions. Reinforcement learning allows the system to learn optimal decision-making strategies by interacting with the environment and receiving feedback in the form of rewards or penalties. This approach enhances the system's ability to make dynamic and context-aware security decisions without requiring manual intervention.

The PPO agent receives inputs from both the anomaly detection and classification models. The Autoencoder provides an anomaly score that indicates the presence of unusual network activity, while the XGBoost classifier identifies whether the traffic is benign or malicious and determines the type of attack. Based on these inputs, the reinforcement learning agent evaluates the current network state and selects the most suitable action. The possible response actions include allowing normal traffic, blocking malicious connections, generating alerts for administrators, or quarantining suspicious devices. This coordinated decision-making process ensures effective threat mitigation and reduces the risk of system compromise.

A key advantage of this module is its ability to adapt to evolving cyber threats. As the system encounters new attack patterns, the PPO agent continuously updates its policy to improve performance. Rewards are assigned for correct decisions, while penalties are given for incorrect responses, guiding the learning process. This continuous improvement mechanism enhances accuracy, reduces false responses, and ensures an intelligent, automated, and scalable cybersecurity solution.

6.4 DECISION ENGINE

The decision engine module acts as the central component of the proposed intrusion detection system, responsible for integrating the outputs of multiple AI models and determining the final security action. Since the system incorporates different models such as the Autoencoder, XGBoost classifier, and reinforcement learning agent, the decision engine plays a crucial role in coordinating their outputs effectively. It ensures that the information generated by each model is analyzed collectively to produce a reliable and accurate threat evaluation. This integration enhances the system's ability to make informed decisions and improves overall detection performance.

The Autoencoder generates an anomaly score that indicates whether the network traffic deviates from normal behavior, helping to identify unknown or suspicious patterns. At the same time, the XGBoost classifier analyzes the network features and classifies traffic as benign or belonging to a specific attack category. These outputs are then provided as input to the reinforcement learning agent, which evaluates the current network condition and suggests the most appropriate response. By combining the strengths of anomaly detection, classification, and intelligent decision-making, the system achieves higher accuracy and reduces false positive rates.Based on the combined inputs, the decision engine calculates a risk level for each network event. The threat levels are categorized as low, medium, high, or critical depending on the severity of the detected activity. After assessing the risk, the decision engine selects suitable response actions such as allowing normal traffic, blocking malicious connections, generating alerts, or quarantining affected systems. This module ensures efficient and timely responses, improving network security and minimizing potential damage.

6.5 OUTPUT / RESPONSE LAYER

The output or response layer is responsible for executing the security actions determined by the decision engine. Once a potential threat is identified and evaluated, this module implements the appropriate response in real time to prevent further damage. The system can perform multiple actions such as allowing legitimate traffic, blocking malicious packets, generating alerts, or quarantining compromised devices. This ensures that detected threats are not only identified but also effectively mitigated, reducing the impact of cyber attacks on the network. The automation of response actions significantly improves the system's ability to react quickly without requiring manual intervention.

Another key function of this module is maintaining detailed logs of all detected events and corresponding system responses. Each intrusion detection event is recorded along with information such as time, type of attack, risk level, and action taken. These logs are stored in structured formats such as CSV files or databases, enabling easy retrieval and analysis. Security analysts can use these logs to study attack patterns, identify vulnerabilities, and evaluate the performance of the detection system. This historical data is also valuable for auditing and improving future security strategies. In addition to logging, the module provides automated alerts and notifications to administrators. Alerts are sent through email or displayed on the monitoring dashboard whenever suspicious activities are detected. The system also generates detailed security reports and supports real-time monitoring of network events. These features enable quick response, enhance situational awareness, and ensure continuous protection of the network environment.

6.6 UI DEVELOPMENT

The user interface (UI) development module focuses on providing an interactive and user-friendly platform for monitoring and managing the intrusion detection system. A web-based dashboard is developed using React to present real-time information about network traffic and detected threats. This dashboard acts as the central visualization layer of the system, allowing administrators to easily observe system activities and security status. The interface is designed with simplicity and clarity in mind, ensuring that even complex network events are displayed in an understandable manner.

The UI displays key information such as detected attacks, anomaly alerts, risk levels, and response actions taken by the system. Various graphical elements such as charts, graphs, and visual indicators are used to represent data effectively. These visualizations help administrators quickly identify unusual patterns and assess the severity of potential threats without analyzing raw data manually. Real-time updates ensure that the dashboard reflects the latest network conditions, enabling continuous monitoring of system performance and security events. In addition to visualization, the UI module provides control functionalities that allow administrators to interact with the system efficiently. Users can start or stop monitoring processes, review historical logs, and generate detailed security reports directly from the dashboard. Real-time notifications highlight critical security events, enabling faster decision-making and response. The interface also supports user-friendly navigation and responsive design for better accessibility. Overall, this module enhances usability, improves situational awareness, and ensures effective management of the intrusion detection system in practical cybersecurity environments.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 COMPARATIVE STUDY

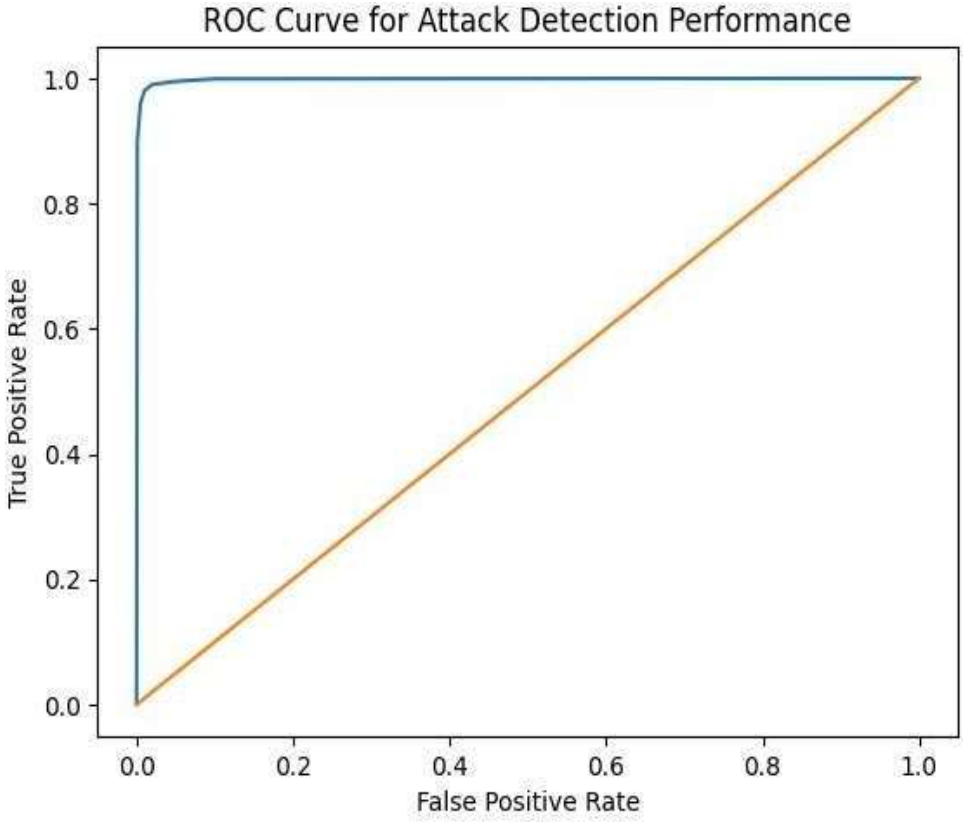

Figure 7 . 1 . ROC Curve Diagram Description

The ROC (Receiver Operating Characteristic) curve illustrates the performance of the proposed intrusion detection system in distinguishing between benign and malicious network traffic. It plots the true positive rate (TPR) against the false positive rate (FPR) at different classification thresholds. In the given diagram, the ROC curve is positioned very close to the top-left corner, which indicates excellent classification performance. This means the system achieves a high detection rate while maintaining a very low false positive rate. The diagonal line in the graph represents a random classifier, and the model's curve remaining significantly above

this line confirms its effectiveness. The sharp rise in the curve at low false positive rates shows that the system can accurately detect attacks with minimal misclassification. Overall, the ROC curve demonstrates that the hybrid AI-based IDS provides strong predictive capability, high reliability, and robustness, making it highly suitable for real-time cybersecurity applications.

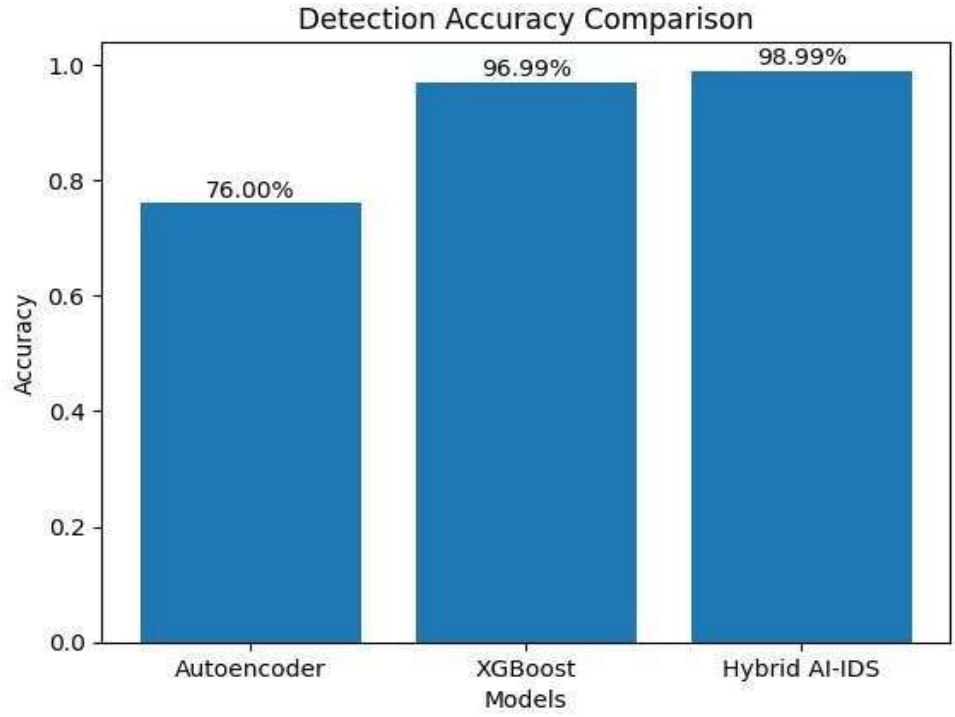

Figure 7 . 2. Detection Accuracy Comparison

The detection accuracy comparison graph highlights the performance of different models used in the intrusion detection system. The Autoencoder model achieves an accuracy of 76%, indicating its ability to detect anomalies in network traffic but with limited classification capability. The XGBoost classifier significantly improves the accuracy to 96.99% due to its supervised learning approach and effective handling of structured data. The proposed Hybrid AI-IDS, which integrates Autoencoder, XGBoost, and reinforcement learning, achieves the highest accuracy of 98.99%.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 CONCLUSION

This project successfully developed a hybrid AI-based intrusion detection and automated response system designed to enhance network security. By integrating multiple artificial intelligence techniques, the system is capable of detecting cyber threats with improved accuracy and efficiency. The use of an Autoencoder model enables the detection of abnormal network behavior, while the XGBoost classifier provides accurate classification of different attack types. This combination improves the reliability of the intrusion detection process.

Another important contribution of the system is the use of reinforcement learning to automate security responses. The Proximal Policy Optimization (PPO) algorithm allows the system to learn optimal response actions based on detected threats. Instead of relying solely on manual intervention, the system can automatically allow safe traffic, block malicious activity, or generate alerts for administrators. This reduces response time and improves the overall security of the network environment.

The system also provides a user-friendly monitoring interface that allows administrators to visualize network activities and security alerts in real time. By combining anomaly detection, attack classification, and intelligent response mechanisms, the proposed system demonstrates the potential of AI-driven solutions in modern cybersecurity environments

8.2 FUTURE ENHANCEMENT

Although the proposed AI-driven intrusion detection system demonstrates strong performance in detecting and responding to cyber threats, several enhancements can be considered to further improve its effectiveness. One important improvement is the integration of additional datasets collected from real-world network environments. Expanding the dataset beyond CIC-IDS2017 will enable the system to learn a wider variety of attack patterns and network behaviors. This will improve its ability to detect emerging and previously unseen threats, making the system more robust and adaptable to dynamic cybersecurity scenarios.

Another potential enhancement involves incorporating advanced deep learning techniques to improve detection accuracy. Models such as Long Short-Term Memory (LSTM) networks and Transformer-based architectures can be used to analyze sequential and time-dependent network traffic patterns. These models are capable of capturing complex temporal relationships, which are essential for identifying sophisticated cyber attacks that evolve over time. Integrating such models into the existing framework can significantly enhance the system's capability to detect advanced persistent threats and reduce false positives. Furthermore, the system can be improved by enhancing scalability and enabling deployment in real-time cloud environments. Integrating the intrusion detection system with cloud-based platforms and Internet of Things (IoT) networks will expand its practical applications. Additionally, implementing continuous and adaptive learning mechanisms will allow the system to update its models automatically as new threats emerge. This ensures long-term efficiency, better adaptability, and improved protection for modern network infrastructures.

.

APPENDIX A

SOURCE CODE

```
import asyncio
import json
import os
import time
import random
from datetime import datetime
from typing import Optional
from collections import deque
from contextlib import asynccontextmanager
import numpy as np
from fastapi import FastAPI, WebSocket, WebSocketDisconnect, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from backend.utils.config_loader import get_config, ROOT_DIR
from backend.models.autoencoder import AnomalyDetector
from backend.models.classifier import SupervisedClassifier
from backend.models.rl_agent import RLAgent
from backend.modules.decision_engine import DecisionEngine
```

```
from backend.modules.response_handler import ResponseHandler

from backend.modules.logger import IDSLogger

from backend.modules.performance import PerformanceEvaluator

from backend.modules.streaming import DataStreamer

from backend.modules.report_generator import generate_attack_report

from backend.modules.email_alert import send_email_alert, build_alert_email_body


detector: Optional[AnomalyDetector] = None

classifier: Optional[SupervisedClassifier] = None

rl_agent: Optional[RLAgent] = None

engine: Optional[DecisionEngine] = None

response_handler: Optional[ResponseHandler] = None

logger: Optional[IDSLogger] = None

evaluator: Optional[PerformanceEvaluator] = None

streamer: Optional[DataStreamer] = None

streaming_task: Optional[asyncio.Task] = None

is_streaming: bool = False

recent_scores = deque(maxlen=200)        # anomaly score time series

recent_events = deque(maxlen=500)        # for live table

traffic_count = 0

anomaly_count = 0

attack_events_buffer = deque(maxlen=2000) # attacks for report generation

@asynccontextmanager

async def lifespan(app: FastAPI):
```

```
global detector, classifier, rl_agent, engine, response_handler, logger, evaluator, streamer
print("\n==========================================")
print("  AI-Driven IDS - Starting Up")
print("==========================================\n")
cfg = get_config()
print("[1/5] Loading Autoencoder...")
detector = AnomalyDetector()
try:
    detector.load()
except Exception as e:
    print(f"  WARNING: Could not load autoencoder: {e}")
    print("  Run train.py first to train models.")
print("[2/5] Loading Supervised Classifier...")
classifier = SupervisedClassifier()
try:
    classifier.load()
except Exception as e:
    print(f"  WARNING: Could not load classifier: {e}")
    print("  Classifier will be skipped in ensemble.")
    classifier = None
print("[3/5] Loading RL Agent...")
rl_agent = RLAgent()
try:
    rl_agent.load()
```

```
def main():
    print("=" * 60)
    print("  AI-Driven IDS - Training Pipeline")
    print("=" * 60)
    start = time.time()
    cfg = get_config()
    ae_cfg = cfg["autoencoder"]
    rl_cfg = cfg["reinforcement_learning"]
    print("\n" + "=" * 60)
    print("  STEP 1: Data Preprocessing")
    print("=" * 60)
    processed_path = str(ROOT_DIR / cfg["data"]["processed_csv"])
    feat_path = str(ROOT_DIR / ae_cfg["feature_columns_path"])
    if os.path.exists(processed_path) and os.path.exists(feat_path):
        print("  Processed data found. Loading...")
        df = pd.read_csv(processed_path)
        with open(feat_path) as f:
            feature_cols = json.load(f)
    else:
        df, feature_cols, scaler = preprocess_and_save(
            sample_size=cfg["data"]["sample_size"]
        )
    print(f"  Dataset: {len(df)} rows, {len(feature_cols)} features")
    print(f"  Labels: {df['Label'].value_counts().to_dict()}")
    print("\n" + "=" * 60)
```

```
print("  STEP 2: Training Autoencoder")
print("=" * 60)
normal = df[df["is_attack"] == 0]
attacks = df[df["is_attack"] == 1]
print(f"  Normal samples: {len(normal)}")
print(f"  Attack samples: {len(attacks)}")
X_normal = normal[feature_cols].values.astype(np.float32)
X_train, X_val = train_test_split(X_normal, test_size=0.2, random_state=42)
ae_cfg["input_dim"] = len(feature_cols)
ae_cfg["encoder_dims"] = _adapt_dims(len(feature_cols), ae_cfg["encoder_dims"])
ae_cfg["decoder_dims"] = list(reversed(ae_cfg["encoder_dims"]))
detector = AnomalyDetector()
detector.build_model(len(feature_cols))
history = detector.train(X_train, X_val)
X_attacks = attacks[feature_cols].values.astype(np.float32)
detector.compute_optimal_threshold(X_val, X_attacks, max_fpr=0.20)
detector.save()
attack_anomalies, attack_scores = detector.predict(X_attacks)
attack_detection_rate = attack_anomalies.mean()
print(f"\n  Attack detection rate: {attack_detection_rate:.2%}")
print(f"  Mean attack score: {attack_scores.mean():.6f}")
val_anomalies, val_scores = detector.predict(X_val)
fpr = val_anomalies.mean()
print(f"  False positive rate (normal val): {fpr:.2%}")
```

```
print("\n" + "=" * 60)
print("  STEP 2.5: Training Supervised Classifier")
print("=" * 60)
clf = SupervisedClassifier()
clf.feature_cols = feature_cols
X_all = df[feature_cols].values.astype(np.float32)
y_all = df["is_attack"].values.astype(np.int32)
label_names_all = df["Label"].values
X_clf_train, X_clf_val, y_clf_train, y_clf_val, lbl_train, lbl_val = \
    train_test_split(X_all, y_all, label_names_all,
                     test_size=0.2, stratify=y_all, random_state=42)
clf_metrics = clf.train(
    X_clf_train, y_clf_train,
    X_clf_val, y_clf_val,
    label_names_train=lbl_train,
    label_names_val=lbl_val,
)
clf.save()
print("\n" + "=" * 60)
print("  STEP 3: Training RL Agent")
print("=" * 60)
n_rl_samples = min(50000, len(df))
n_attack_rl = min(len(attacks), n_rl_samples // 2)
n_normal_rl = n_rl_samples - n_attack_rl
```

```
rl_normal = normal.sample(n=min(n_normal_rl, len(normal)), random_state=42)
rl_attacks = attacks.sample(n=min(n_attack_rl, len(attacks)), random_state=42)
rl_df = pd.concat([rl_normal, rl_attacks]).sample(frac=1, random_state=42)
X_rl = rl_df[feature_cols].values.astype(np.float32)
y_rl = rl_df["is_attack"].values.astype(np.int32)
_, rl_scores = detector.predict(X_rl)
agent = RLAgent()
feature_dim = min(10, len(feature_cols))
agent.train(X_rl, y_rl, rl_scores,
            feature_dim=feature_dim,
            total_timesteps=rl_cfg["total_timesteps"])
agent.save()
elapsed = time.time() - start
print("\n" + "=" * 60)
print("  TRAINING COMPLETE")
print("=" * 60)
print(f"  Total time: {elapsed:.1f}s ({elapsed/60:.1f} min)")
print(f"  Autoencoder: {ae_cfg['model_path']}")
print(f"  Classifier: {cfg.get('classifier', {}).get('model_path', 'classifier.pkl')}")
print(f"  RL Agent: {rl_cfg['model_path']}")
print(f"  AE Attack detection rate: {attack_detection_rate:.2%}")
print(f"  AE False positive rate: {fpr:.2%}")
print(f"  XGB F1: {clf_metrics['f1']:.4f}  Recall: {clf_metrics['recall']:.2%}  FPR: {clf_metrics['fpr']:.2%}")
print(f"\n  Next: Run the server with:")
print(f"    python run_server.py")
```

```
      <div className="lm-value">{value}</div>
      <div className="lm-label">{label}</div>
    </div>
  );
}
function SOCDashboard({ onBack }) {
  const [status, setStatus] = useState(null);
  const [metrics, setMetrics] = useState(null);
  const [scores, setScores] = useState([]);
  const [events, setEvents] = useState([]);
  const [isStreaming, setIsStreaming] = useState(false);
  const [connected, setConnected] = useState(false);
  const [toast, setToast] = useState(null);
  const [loading, setLoading] = useState({});
  const wsRef = useRef(null);
  const toastTimer = useRef(null);
  const showToast = useCallback((message, type = 'info') => {
    setToast({ message, type });
    if (toastTimer.current) clearTimeout(toastTimer.current);
    toastTimer.current = setTimeout(() => setToast(null), 4000);
  }, []);
  useEffect(() => {
    let ws;
    let reconnectTimer;
```

```
const connect = () => {
  ws = new WebSocket(WS_URL);
  wsRef.current = ws;
  ws.onopen = () => setConnected(true);
  ws.onclose = () => {
  setConnected(false);
    reconnectTimer = setTimeout(connect, 3000);
  };
  ws.onerror = () => setConnected(false);
  ws.onmessage = (event) => {
    try {
      const data = JSON.parse(event.data);
      if (data.metrics) setMetrics(data.metrics);
      if (data.recent_scores) setScores(prev => {
        const combined = [...prev, ...data.recent_scores];
        return combined.slice(-200);
      });
      if (data.recent_events) setEvents(prev => {
const incoming = data.recent_events.reverse();
const merged = [...incoming, ...prev];
const unique = Array.from(
  new Map(merged.map(e => [e.timestamp, e])).values()
);
return unique.slice(0, 100);   // ← change 100 to 50/150 as needed
```

```
});
        setIsStreaming(data.is_streaming || false);
        setStatus({
          traffic_count: data.traffic_count,
          anomaly_count: data.anomaly_count,
          is_streaming: data.is_streaming,
          handler_stats: data.handler_stats,
          timestamp: data.timestamp,
        });
      } catch (e) { /* ignore parse errors */ }
    };
  };
  connect();
  return () => {
    if (ws) ws.close();
    if (reconnectTimer) clearTimeout(reconnectTimer);
  };
}, []);
return (
  <div className="app-container">
    {/* Header */}
    <header className="header">
      <div className="header-left">
        <button className="back-btn" onClick={onBack} title="Back to Home">
```

```
    <ChevronRight size={16} style={{ transform: 'rotate(180deg)' }} /> Home
  </button>
  <div className="header-logo"><Shield size={20} /></div>
  <div>
    <div className="header-title">AI-Driven IDS</div>
    <div className="header-subtitle">Security Operations Center</div>
  </div>
</div>
<div className="header-right">
  <button
    onClick={() => setDemoMode(!demoMode)}
    className="btn btn-outline"
    style={{
      padding: '6px 12px',
      display: 'flex',
      alignItems: 'center',
      gap: '6px',
      borderColor: demoMode ? '#06b6d4' : '#374151',
      color: demoMode ? '#06b6d4' : '#9ca3af',
      background: demoMode ? 'rgba(6, 182, 212, 0.1)' : 'transparent',
      fontSize: '12px',
      marginRight: '10px'
    }}
    title={demoMode ? "Demo Mode Active (Visual Override)" : "Enable Demo Mode"}
```

```
    >
      {demoMode ? <ToggleRight size={16} /> : <ToggleLeft size={16} />}
    </button>
    <div className={`status-badge ${connected ? 'online' : 'offline'}`}>
      <span className={`status-dot ${connected ? 'online' : 'offline'}`} />
      {connected ? 'Connected' : 'Disconnected'}
    </div>
    {isStreaming && (
      <div className="status-badge online">
        <Radio size={12} /> Live
      </div>
    )}
  </div>
</header>
```

APPENDIX B

SCREENSHOTS

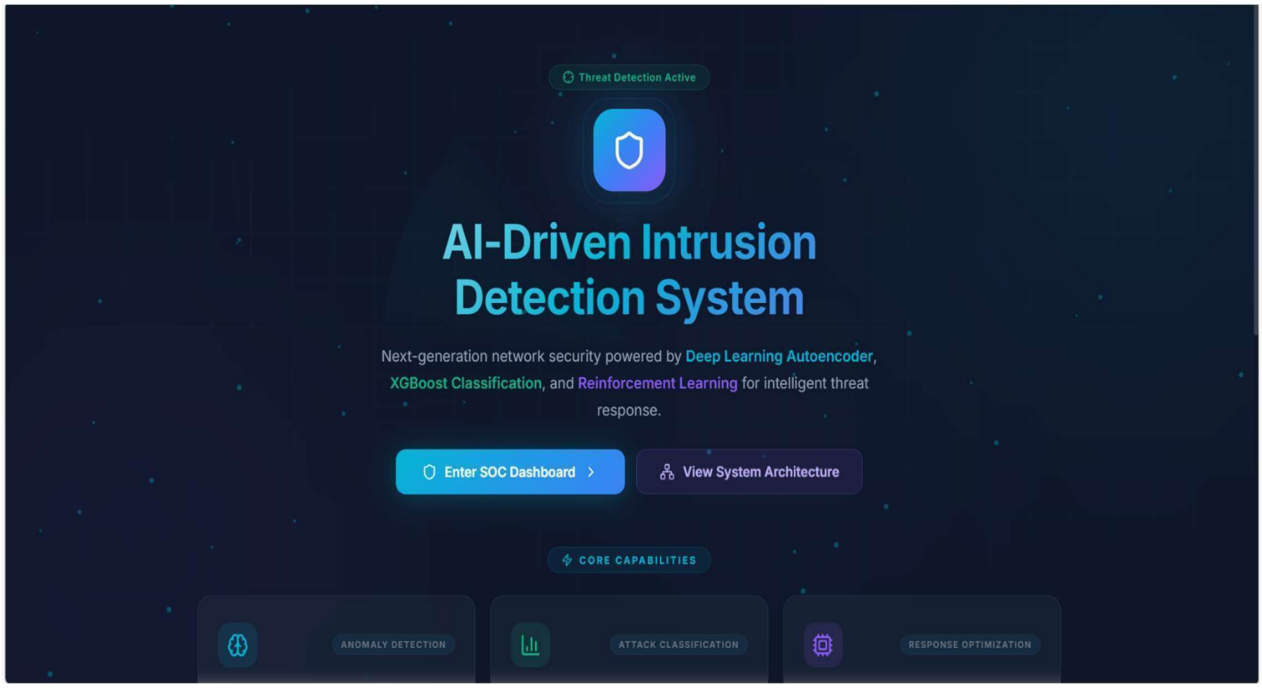

Figure .B.1 Home Page

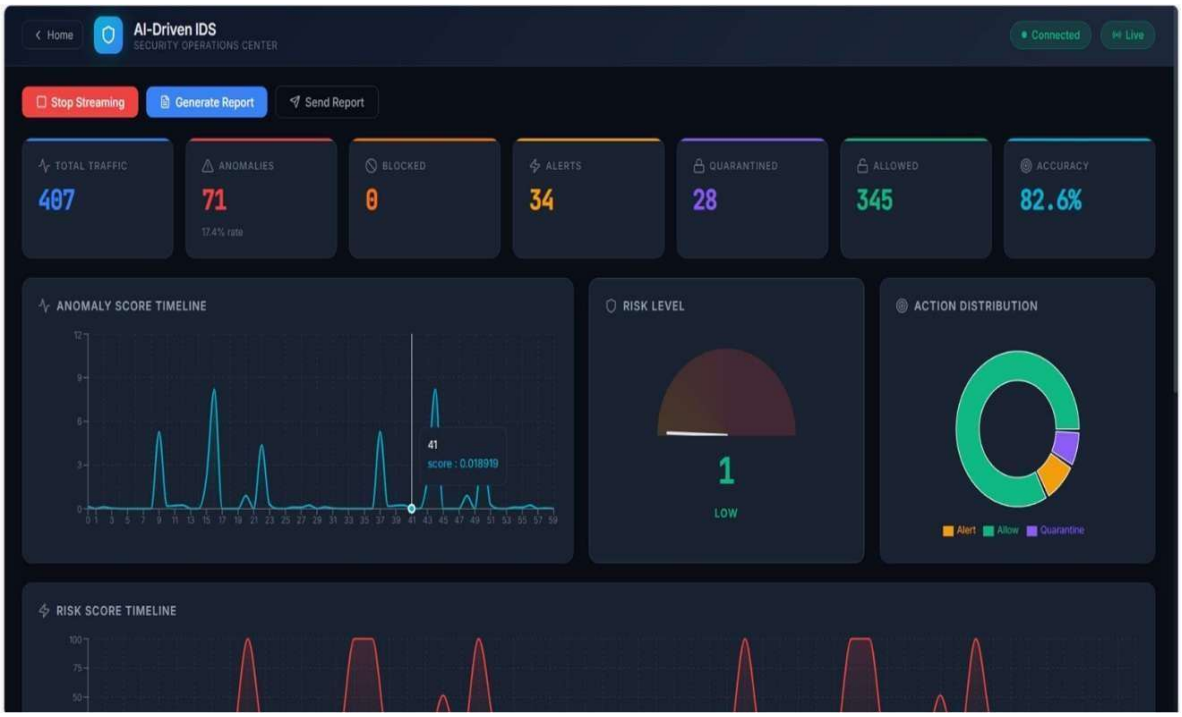

Figure.B.2 IDS Dashboard

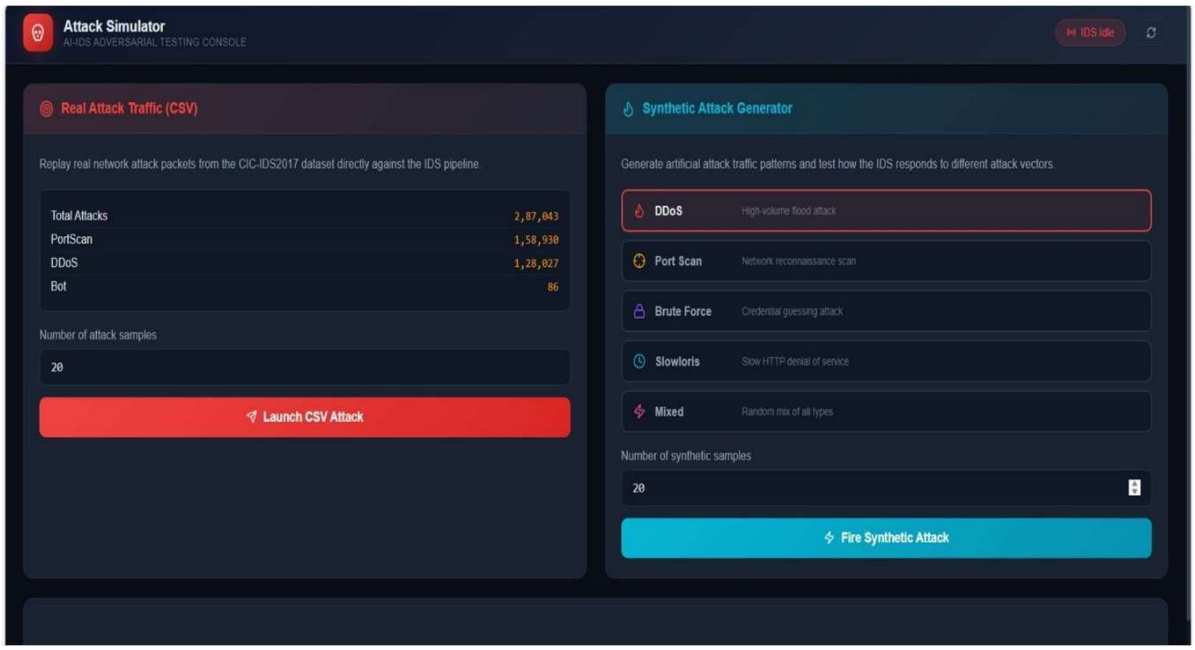

Figure.B.3 Attacker Interface

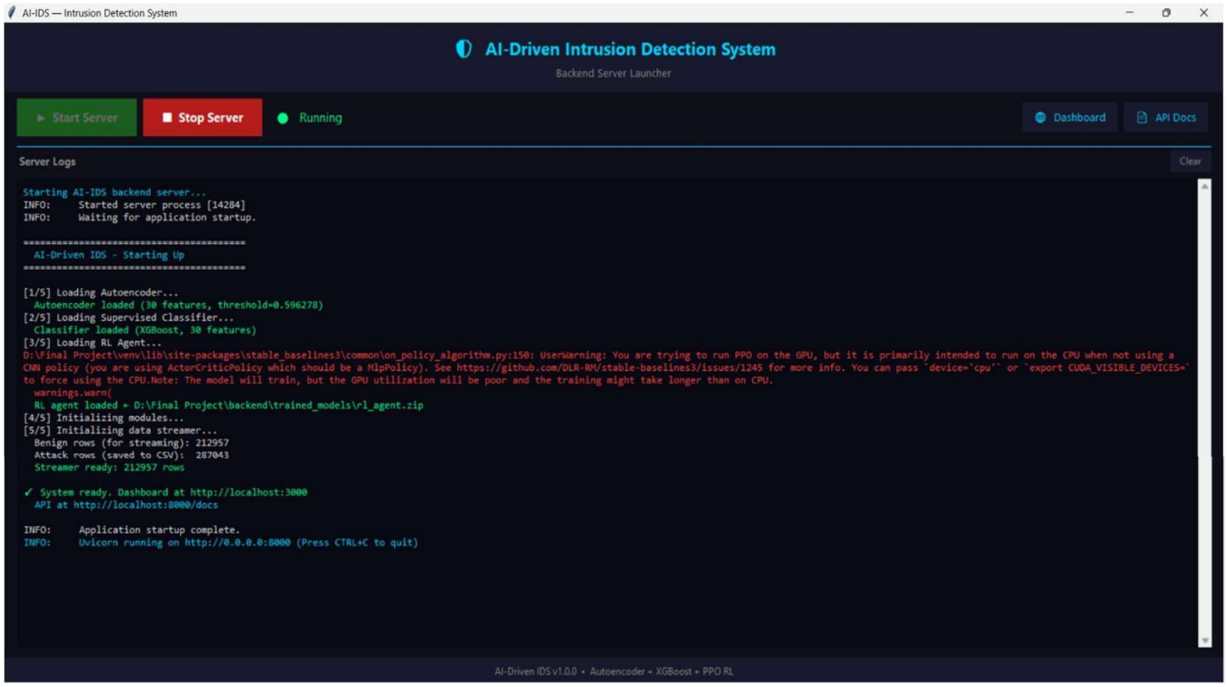

Figure.B.4 Backend Launcher

REFERENCES

1. Agarwal, S., Farid, H. Detecting from Phoneme–Viseme Mismatches. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2025
2. Chandrasekaran, V., Rao, P. Audio Forensics for Detecting Voice Cloning Using Spectral Feature Analysis. ACM Multimedia Systems Conference. 2024
3. Chen, Y., Zhang, W. Face Manipulation Detection via Convolutional Neural Networks and Attention Fusion. International Journal of Computer Vision. 2023
4. Dalmia, S., Das, S. Hybrid Multimodal Transformer Architecture for Deepfake Detection in Video and Audio. IEEE Transactions on Information Forensics and Security. 2023
5. Güera, D., Delp, E. DeepFake Video Detection Using Recurrent Neural Networks. IEEE International Conference on Advanced Video and Signal-Based Surveillance. 2023
6. Korshunov, P., Marcel, S. DeepFakes: A New Threat to Face Recognition Assessment and Detection. EPFL Technical Report. 2022
7. Mittal, T., Bhattacharya, U. Emotional Speech-Based Detection of AI Synthesized Audio Embeddings. Interspeech Conference,2022.
8. Rossler, A., Cozzolino, D., Riess, C. FaceForensics++: Learning to Detect Manipulated Facial Images. IEEE International Conference on Computer Vision (ICCV). 2021
9. Singh, R., Kumar, A. EfficientNet-Based Deepfake Image Classification for Cybersecurity Applications. Journal of Digital Forensics and Cyber Threat Intelligence. 2020
10. Zhou, P., Han, X., Morariu, V. Two Stream Neural Networks for Tampered Face Detection. IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). 2020